Week 6 : K-Nearest Neighbor (k-NN)

Introduction

K-Nearest Neighbor is one of the simplest Machine Learning algorithms based on Supervised Learning technique. K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories. K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm. K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems. K-NN is a **non-parametric algorithm**, which means it does not make any assumption on underlying data. It is also called a **lazy learner algorithm** because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset. KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

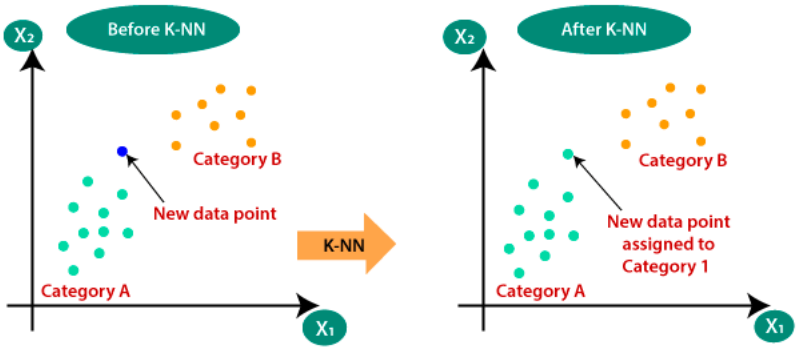
Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.



Figure 1 K-Nearest Neighbor Classifier

Need of a K-Nearest Neighbor Algorithm

Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x1, so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



Working of a K-NN

Step 1: Selecting the optimal value of K

- K represents the number of nearest neighbors that needs to be considered while making prediction.

Step 2: Calculating distance

- To measure the similarity between target and training data points Euclidean distance is used.
- Distance is calculated between data points in the dataset and target point.
- Eg: Euclidean distance, Manhattan distance, Minkowski ditance

Step 3: Finding Nearest Neighbors

- The k data points with the smallest distances to the target point are nearest neighbors.

Step 4: Voting for Classification or Taking Average for Regression

Mathematical Distance Metrics

The choice of "distance" determines how neighborhood similarity is quantified.

- **Euclidean Distance** (L2 Norm): Best for dense, continuous geometric features.

$$d(x,q) = \sqrt{\sum_{j=1}^{n}(x_j - q_j)^2}$$

- **Manhattan Distance** (L1 Norm): Best when features are grid-like or high-dimensional.

$$d(x,q) = \sum_{j=1}^{n}|x_j - q_j|$$

- **Minkowski Distance**: Generalization of L1 and L2 (p=1 is Manhattan, p=2 is Euclidean).

$$d(x,q) = \left(\sum_{j=1}^{n}|x_j - q_j|^p\right)^{\frac{1}{p}}$$

Pseudo Code of KNN

We can implement a KNN model by following the below steps:
1. Load the data
2. Initialise the value of k
3. For getting the predicted class, iterate from 1 to total number of training data points
a. Calculate the distance between test data and each row of training dataset. Here we will use Euclidean distance as our distance metric since it's the most popular method. The other distance function or metrics that can be used are Manhattan distance, Minkowski distance, etc. If there are categorical variables, hamming distance can be used.
b. Sort the calculated distances in ascending order based on distance values
c. Get top k rows from the sorted array
d. Get the most frequent class of these rows
e. Return the predicted class

Exercises

Q-1. A Classification of Fruits

You are provided with a dataset of fruits. Each fruit is characterized by two features: weight (in grams) and sweetness level (on a scale of 1 to 10). You want to classify a new fruit as either an "Apple" or an "Orange" based on these features using the KNN algorithm.

Fruit ID	Weight (grams)	Sweetness Level	Label (Fruit Type)
1	180	7	Apple
2	200	6	Apple
3	150	4	Orange
4	170	5	Orange
5	160	6	Apple
6	140	3	Orange

Tasks:

1. Implement the KNN algorithm manually with k=3 to classify a new fruit with a weight of 165 grams and sweetness level of 5.5.
2. Calculate the Euclidean, Manhattan, and Minkowski distances between the new fruit and all the existing fruits in the dataset. Finally compare the calculated distances.
3. Based on the k-nearest neighbors, determine the label for the new fruit.
4. What is the effect of choosing different values of k (e.g., k=1, k=5) on the classification result?
5. Implement the above using function python program without using scikit learn library.
6. Plot the given samples, the Apple in Red color and the Orange in orange color. Also draw the decision boundary.

Q-1. B Classification of Fruits

Implement the Python code for Q-1. A using the scikit-learn library. Plot the given samples, using red for "Apple" and orange for "Orange." Also, plot the decision boundary. Calculate the distances using Euclidean, Manhattan, and Minkowski metrics, and compare the results.